

Mini-Lakehouse Distribuido para la Universidad de Antioquia

Integrantes

- Manuela Zapata Atencia
 - Oscar Danilo Granada Giraldo
 - Arley Burítica García
-

1. Introducción

La Universidad de Antioquia enfrenta problemas de concurrencia durante los periodos de matrícula académica, especialmente en cursos de alta demanda donde miles de estudiantes intentan registrarse simultáneamente.

El sistema monolítico actual presenta:

- bloqueos transaccionales;
- timeouts;
- alta latencia;
- contención sobre cursos hotspot;
- problemas de escalabilidad durante los picos de inscripción.

Con el objetivo de mejorar el procesamiento analítico y soportar escenarios distribuidos, se diseñó un Mini-Lakehouse local basado en arquitectura Bronze, Silver y Gold usando DuckDB y archivos Parquet.

2. Problema identificado

El principal cuello de botella identificado no corresponde al estudiante (`student_id`), sino a la alta contención concurrente sobre determinados cursos (`course_id`) durante ventanas temporales específicas.

Esto genera:

- retries masivos;
- agotamiento rápido de cupos;
- timeouts;
- latencia elevada;
- efecto thundering herd.

Por esta razón, el diseño se enfocó en modelar:

- intentos de inscripción;
- resultados operacionales;
- comportamiento temporal;

- métricas de concurrencia.

3. Arquitectura propuesta

La solución implementa una arquitectura Lakehouse local compuesta por tres capas:

Bronze

Almacena datos crudos en formato Parquet:

- estudiantes;
- cursos;
- inscripciones.

Silver

Realiza:

- limpieza;
- tipificación;
- validaciones;
- eliminación de duplicados;
- control de calidad.

Gold

Implementa:

- modelo dimensional;
- star schema;
- surrogate keys;
- métricas analíticas;
- dimensiones históricas.

DuckDB se utiliza como motor OLAP para consultas analíticas sobre archivos Parquet.

4. Modelo dimensional

Tabla de hechos

fact_inscripciones

Granularidad:

1 fila = 1 intento de inscripción de un estudiante a un curso en un instante específico.

Métricas

- duracion_ms
 - retries
 - timeouts
 - inscripcion_exitosa
 - fue_rechazada
-

Dimensiones

dim_tiempo

Permite análisis temporal:

- hora;
- fecha;
- franja horaria;
- día;
- mes.

dim_estudiante

Contiene:

- facultad;
- semestre;
- sede;
- historial SCD Tipo 2.

dim_curso

Modela:

- cursos hotspot;
- facultad;
- cupos.

dim_sede

Permite analizar:

- latencia regional;
- conectividad.

dim_resultado

Clasifica:

- exitosa;
- retry;
- timeout;
- cupo agotado.

5. Generación de datos sintéticos

Se generaron:

- 200.000 estudiantes;
- 500 cursos;
- 500.000 intentos de inscripción.

El sistema simula:

- hotspots;
- retries;
- timeouts;
- alta concurrencia;
- diferencias regionales de latencia.

Los cursos hotspot representan escenarios reales de contención.

6. Pipeline implementado

El pipeline completo fue automatizado mediante `main.py`.

Flujo

```
graph TD
    A[Fuente Operacional] --> B[Bronze]
    B --> C[Silver]
    C --> D[Gold]
    D --> E[DuckDB]
    E --> F[Consultas Analíticas]
```

7. Consultas analíticas

Se implementaron consultas para:

- inscripciones por hora;
- cursos con mayor contención;
- latencia por sede;

- resultados operacionales;
- tasa de éxito por facultad.

Estas consultas permiten identificar:

- picos de demanda;
- hotspots;
- comportamiento transaccional;
- problemas regionales.

8. Benchmark

Se comparó:

- pandas sobre Silver;
- DuckDB sobre Gold.

Resultados obtenidos

Motor	Tiempo
Pandas	120.89 ms
DuckDB	71.65 ms

Mejora observada

DuckDB fue 1.69x más rápido.

Esto demuestra las ventajas de:

- almacenamiento columnar;
- Parquet;
- workloads OLAP especializados.

9. Consideraciones distribuidas

El diseño considera:

- hotspots por curso;
- concurrencia masiva;
- retries;
- timeouts;
- latencia regional;
- separación OLTP/OLAP.

El sistema evita analizar únicamente por `student_id`, ya que el verdadero cuello de botella ocurre sobre `course_id`.

10. Conclusiones

La arquitectura Lakehouse permitió:

- separar procesamiento operacional y analítico;
- optimizar consultas agregadas;
- mejorar rendimiento analítico;
- modelar concurrencia realista;
- implementar un pipeline reproducible.

DuckDB demostró mejoras importantes frente a cargas analíticas tradicionales sobre datos operacionales.

El modelo dimensional facilita el análisis histórico y operacional de los procesos de matrícula académica.

11. Tecnologías utilizadas

- Python
- pandas
- DuckDB
- Parquet
- Graphviz
- NumPy
- Faker